

# 崩れゆく城の設計図：Three.js で創るインタラクティブな世界



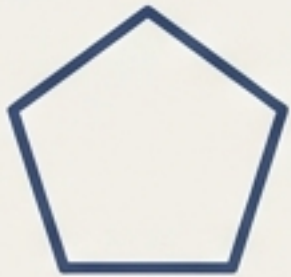


# 構築する世界の構成要素

このプロジェクトは、単一のシーンに複数のインタラクティブ要素を組み合わせることで、没入感のある体験を構築します。主要な要素は以下の通りです。



- **シーン (Scene)** : 石造りの壁、高い天井、そして柱によって構成される「城」の内部空間。



- **床 (Floor)** : 踏むと1秒後に落下する、100x100の正五角形タイル。カラフルなデザインが特徴。



- **プレイヤー (Player)** : 一人称視点（胸の高さのカメラ）で、WASD移動とマウスによる視点操作が可能。歩行・走行アニメーションも実装。



- **ギミック (Gimmicks)** : プレイヤーの進行を妨害する動的な障害物。巨大な振り子などが雰囲気を高める。



- **物理挙動 (Physics)** : 重力と、落下する床の挙動をシミュレート。



# 成功への道筋：最小構成から始める建築アプローチ

傑作は一度には生まれません。私たちは、まず「プレイ可能なコア」を確立し、そこから段階的に機能とディテールを積み上げていきます。この順序が、プロジェクトを管理しやすくし、モチベーションを維持する鍵となります。

1

## プレイヤーの確立 (Establish Player)

一人称視点での移動 (First-person movement)

2

## コアメカニクスの実装 (Implement Core Mechanic)

床を踏む判定と落下の実装 (Detecting and dropping the floor)

3

## 世界の拡張 (Expand the World)

床を100x100に敷き詰める (Tiling the 100x100 floor)

4

## 雰囲気醸成 (Create Atmosphere)

城のビジュアルとライティング (Castle visuals and lighting)

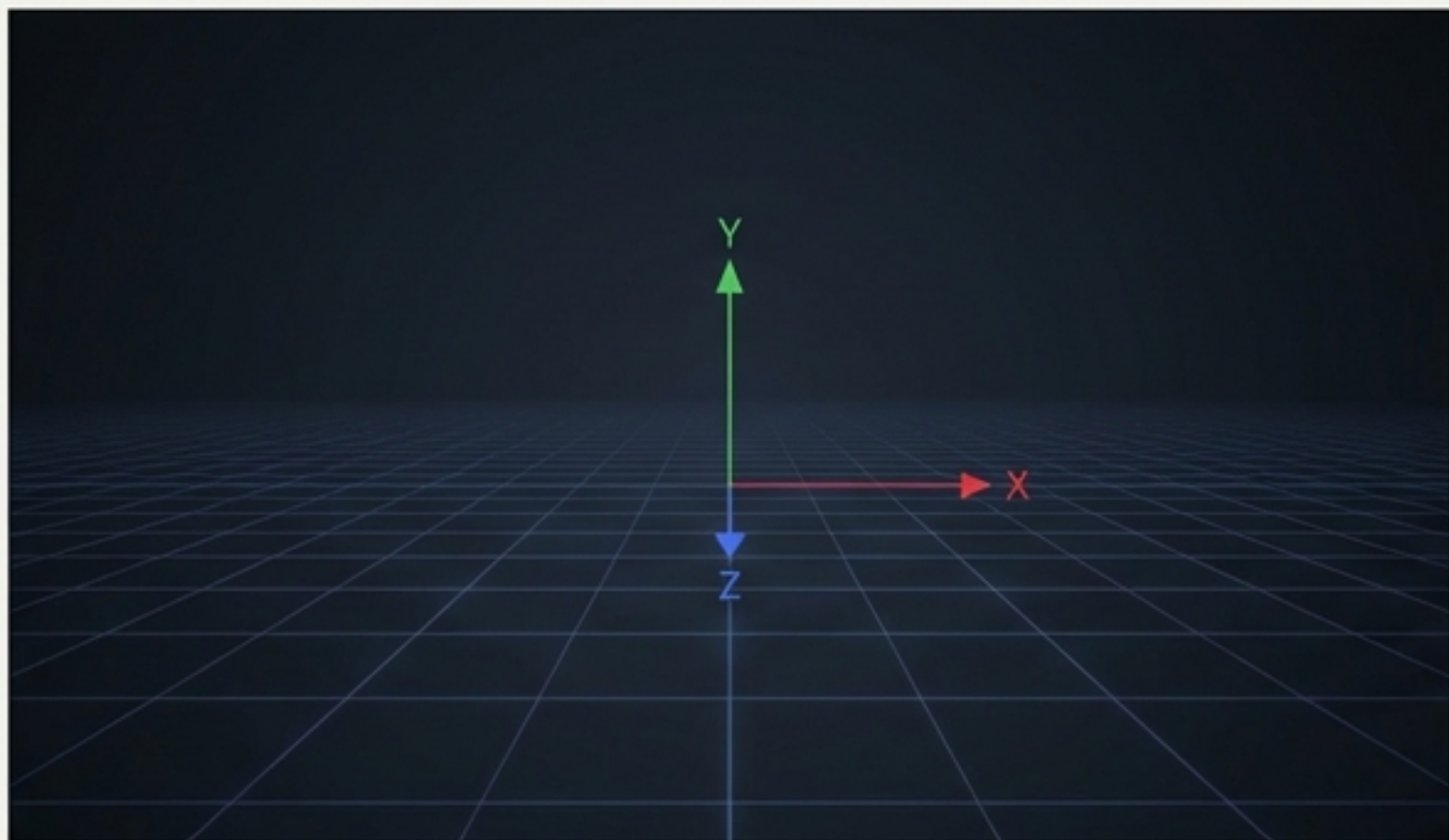
5

## 仕上げ (Add Polish)

妨害ギミックとプレイヤーの手の表示 (Obstacles and player hands)



# 基礎工事①：プレイヤーの視点と操作を実装する



## 一人称視点の実現 (Achieving a First-Person View)

カメラをプレイヤーの「胸の高さ」に設定することで、リアルな没入感を生み出します。

```
camera.position.y = 1.4;
```

## 自由な視点移動 (Free Look Controls)

マウス操作による視点移動には`PointerLockControls`が定番です。カーソルをロックし、シームレスな操作を提供します。

```
import { PointerLockControls } from
'three/addons/controls/PointerLockControls.js';

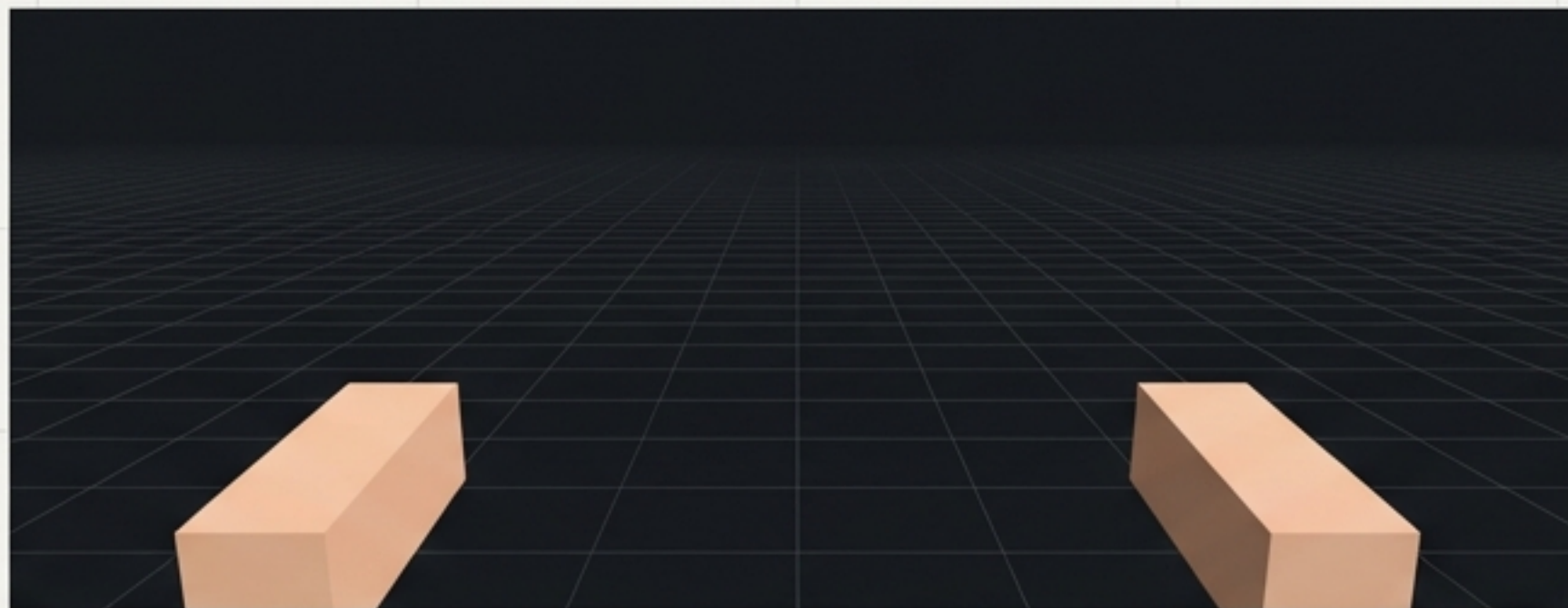
// ... scene setup ...

const controls = new PointerLockControls(camera,
document.body);
scene.add(controls.getObject());
```



## 基礎工事②：プレイヤーの「存在」を表現する

画面に手を表示することで、プレイヤーは自身がその世界に存在していると強く感じることができます。シンプルなジオメトリをカメラに追従させ、移動速度に応じて揺れのアニメーションを追加します。



### Hand Creation Code

```
// A simple hand model
const handGeo = new THREE.BoxGeometry(0.1, 0.3, 0.1);
const handMat = new THREE.MeshStandardMaterial({ color: 0xffccaa });
const leftHand = new THREE.Mesh(handGeo, handMat);
const rightHand = new THREE.Mesh(handGeo, handMat);

// Attach hands to the camera
camera.add(leftHand, rightHand);
leftHand.position.set(-0.3, -0.3, -0.5);
rightHand.position.set(0.3, -0.3, -0.5);
```

### 歩行・走行アニメーション (Walk/Run Animation)

- `Math.sin()` を利用して、自然な手の揺れを表現。
- `speed` パラメータで歩行と走行の揺れの速さを切り替えます。

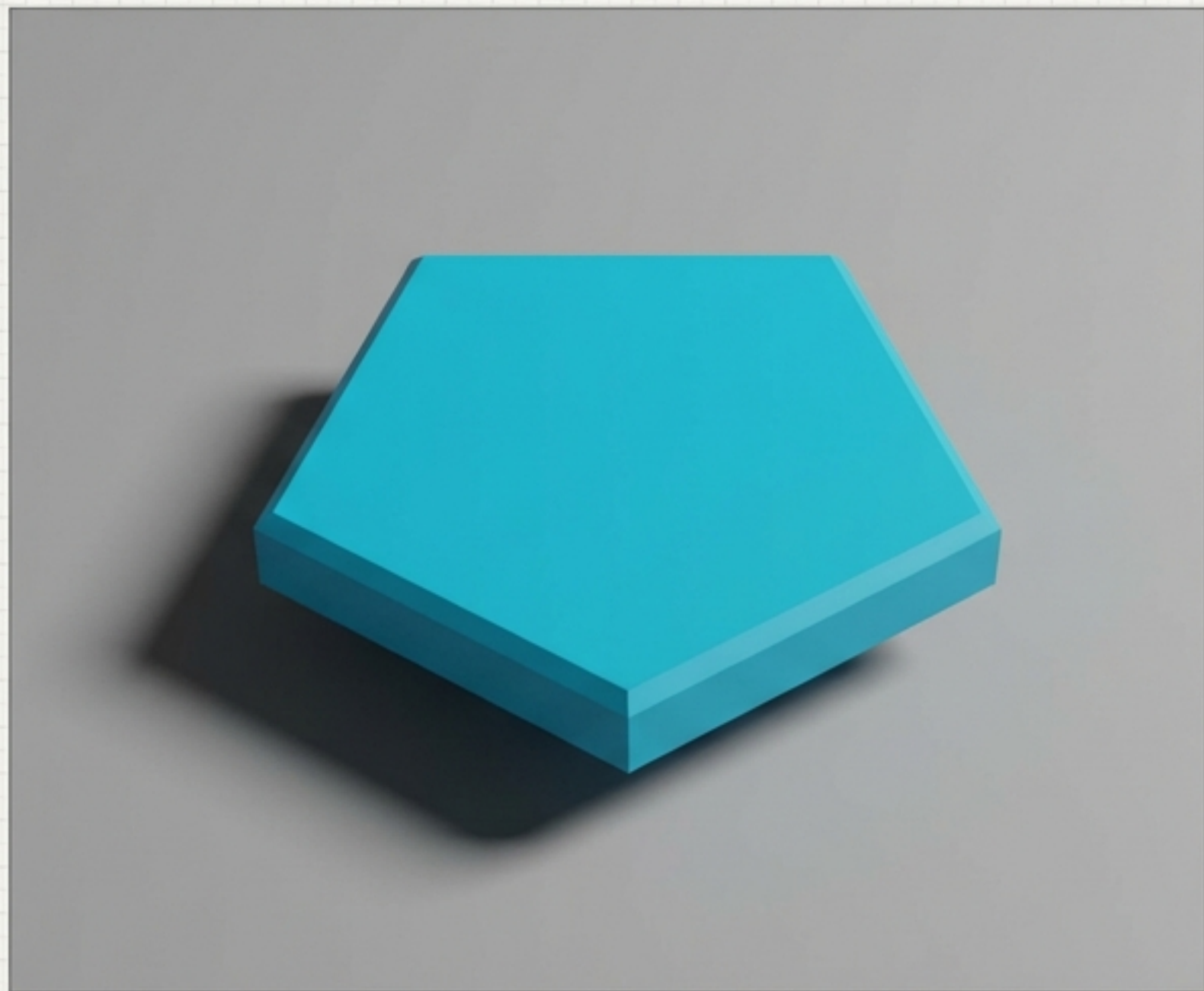
```
function updateHands(speed) {
  time += speed;
  const swing = Math.sin(time) * 0.15;
  leftHand.rotation.x = swing;
  rightHand.rotation.x = -swing;
}

// Walking speed
updateHands(0.1);
// Running speed
updateHands(0.25);
```



# 整地①：ゲームの核となる最初のタイルを創る

床は、この世界の最も重要なインタラクション要素です。`THREE.Shape` を使用してカスタムの正五角形ジオメトリを作成し、ランダムな色を割り当てることで、視覚的に面白い基盤を築きます。



## 正五角形ジオメトリの生成 (Generating Pentagon Geometry)

```
const shape = new THREE.Shape();
const radius = 1;
for (let i = 0; i < 5; i++) {
  const angle = (i / 5) * Math.PI * 2;
  const x = Math.cos(angle) * radius;
  const y = Math.sin(angle) * radius;
  if (i === 0) shape.moveTo(x, y);
  else shape.lineTo(x, y);
}
shape.closePath();
const geometry = new THREE.ShapeGeometry(shape);
```

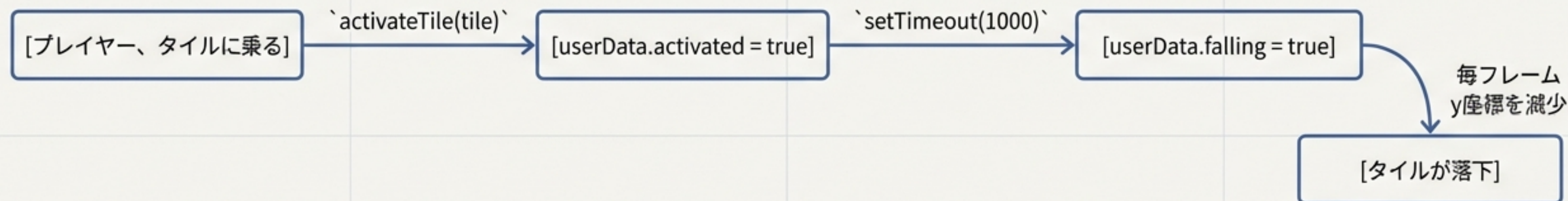
## カラフルなマテリアル (Colorful Material)

```
const material = new THREE.MeshStandardMaterial({
  color: new THREE.Color(Math.random(), Math.random(),
    Math.random())
});
```



## 整地②：タイルに「命」を吹き込む落下ロジック

プレイヤーのアクションに応じて世界が変化する、その核心ロジックです。`userData`プロパティで各タイルの状態（作動済みか、落下中か）を管理し、`setTimeout`で落下までの遅延を実装します。



### 作動ロジック (Activation Logic)

```
function activateTile(tile) {  
  if (tile.userData.activated) return;  
  tile.userData.activated = true;  
  setTimeout(() => {  
    tile.userData.falling = true;  
  }, 1000); // 1-second delay  
}
```

### 毎フレームの更新 (Per-Frame Update)

```
// In the animation loop  
if (tile.userData.falling) {  
  tile.position.y -= 0.1;  
}
```



# 構造躯体：100x100の広大な世界を敷き詰める



1つのタイルを、今度は広大なフィールドへと拡張します。ネストした`for`ループを使い、100x100の範囲にタイルを配置します。正五角形は完全な平面充填ができないため、「見た目重視で少し重なる」アプローチを採用し、隙間のない床を実現します。

```
// The tiles array will store all tile meshes
const tiles = [];

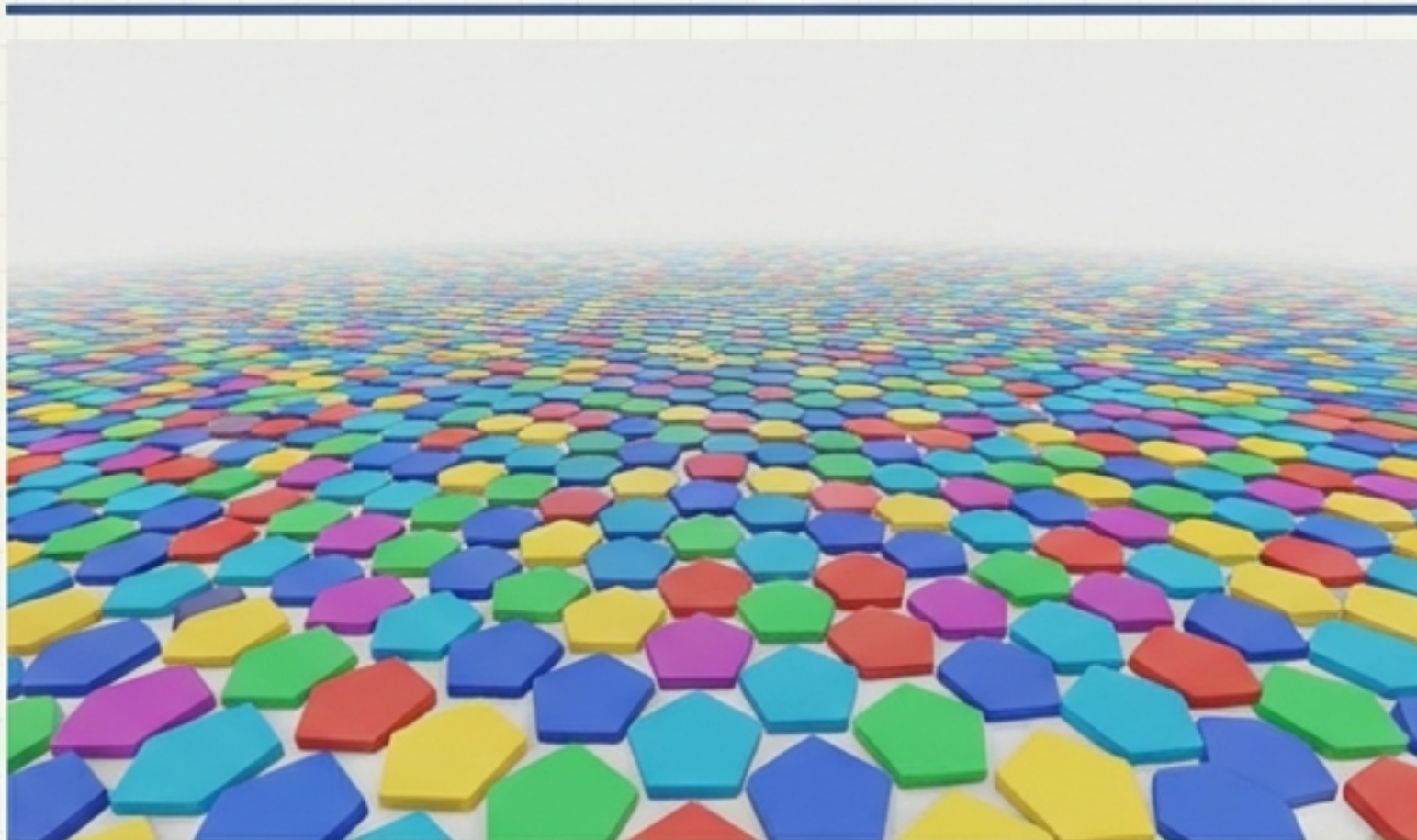
for (let x = -50; x < 50; x += 2.2) { // x-axis loop
  for (let z = -50; z < 50; z += 2.2) { // z-axis loop
    const tile = new THREE.Mesh(geometry, material.clone());
    tile.rotation.x = -Math.PI / 2; // Orient flat
    tile.position.set(x, 0, z);
    // Initialize custom data for state management
    tile.userData = { activated: false, falling: false };
    scene.add(tile);
    tiles.push(tile);
  }
}
```



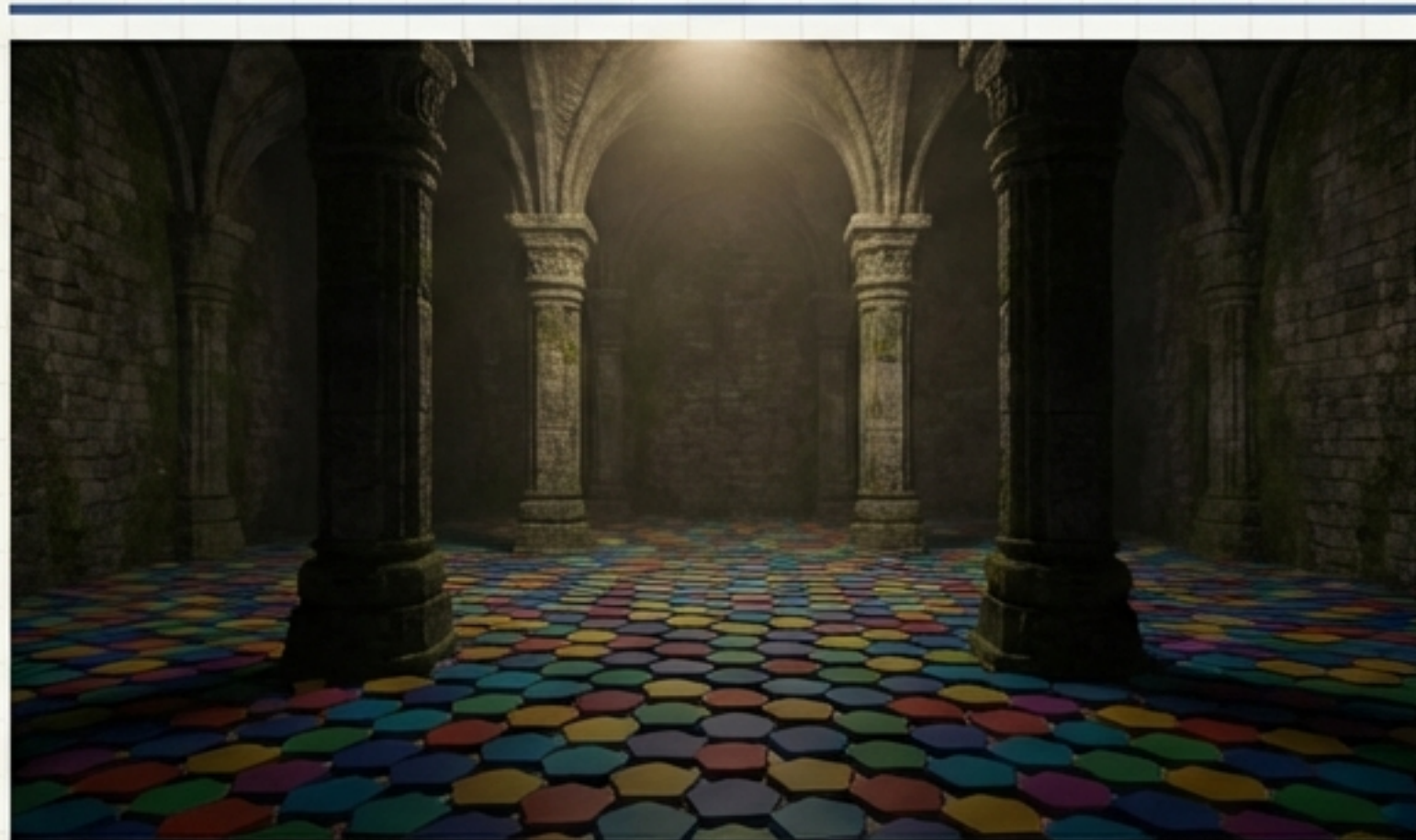
# 仕上げ①：無機質な空間から「城」を建築する

「城感」を出すためには、いくつかの重要な要素があります。石のテクスチャ、高い天井、そして空間に奥行きを与える柱。これらに、一点からの薄暗いライティングを組み合わせることで、雰囲気は劇的に変わります。

Before



After



- 石のようなテクスチャ
- 高い天井
- 巨大な柱
- 暗めのライティング

## Lighting Setup

```
const light = new THREE.PointLight(0xffeecc, 1); // Warm, dim light
light.position.set(0, 10, 0); // Positioned high above
scene.add(light);
```



## 仕上げ②：ディテールで物語を語る

優れた環境は、プレイヤーに進むべき道を示し、物語を暗示します。揺らめく松明の光、天窓から差し込む月明かり、そして最終的なゴールとなる巨大な門。これらのディテールが、ただの空間を「ダンジョン」へと昇華させます。

- 照明演出  
揺らめく松明と月明かりのようなトップライト
- 建築的ディテール  
高い位置にある窓と出口となる巨大な門
- 目的の提示  
門はプレイヤーの視線を自然に引きつけ、進むべき方向を暗示する





## 仕上げ③：テーマに沿った動的ギミックの追加

静的な世界に動きを加えることで、緊張感と挑戦が生まれます。単純な「動く柱」から始め、それをファンタジーの世界観に合わせた「巨大な振り子」へと進化させることで、ギミック自体が世界の物語の一部となります。

### 動く柱 (Moving Pillar)



「`Math.sin()`」を利用して、柱を周期的に左右に動かす。

```
// Obstacle moves back and forth
obstacle.position.x = Math.sin(clock.getElapsedTime()) * 5;
```

### 巨大な振り子 (Giant Pendulum)



- 単なる棒ではなく、天井から吊るされた脅威的な存在。
- プレイヤーの進路を物理的に妨害する。
- 「城」や「ダンジョン」の雰囲気を一層強める。



# 設計資料室：プロの実践から学ぶ

これまでの知識で、見た目と基本的な機能を持つシーンは完成しました。しかし、これを「優れたゲーム」にするためには、さらに3つの重要な柱について深く理解する必要があります。



## 1. 判定方法 (Detection Method)

Raycasterによる正確な衝突判定



## 2. パフォーマンス (Performance)

InstancedMeshによる描画の最適化



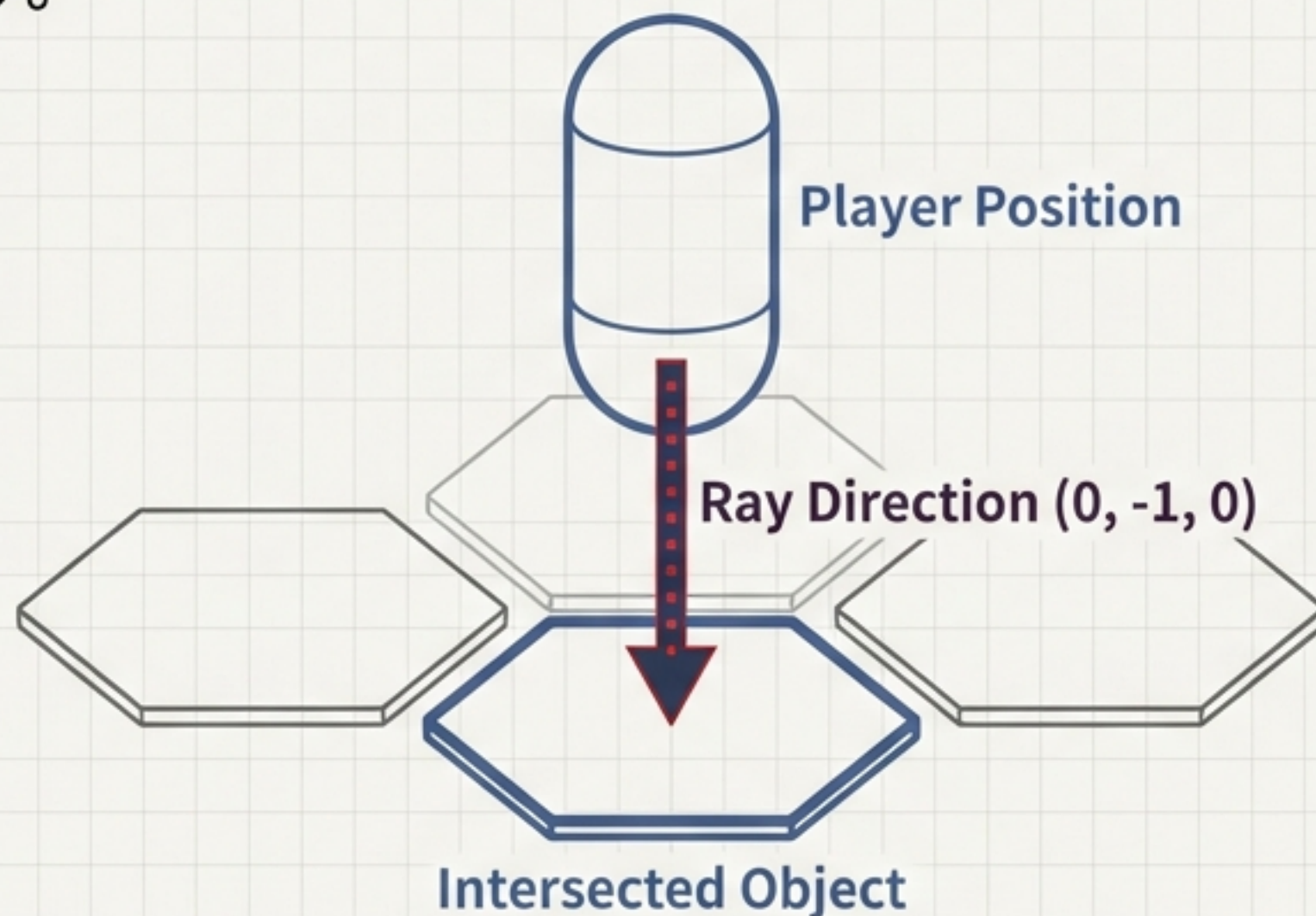
## 3. ゲーム性 (Game Design)

プレイヤーを惹きつけるルール設計



# Annex A: 正確な判定 — Raycasterで足元の床を特定する

プレイヤーが「どのタイルを踏んでいるか」を正確に知る必要があります。これを実現するのが`Raycaster`です。プレイヤーの足元から真下に向けて光線（Ray）を放ち、最初に交差したオブジェクト（床タイル）を特定します。



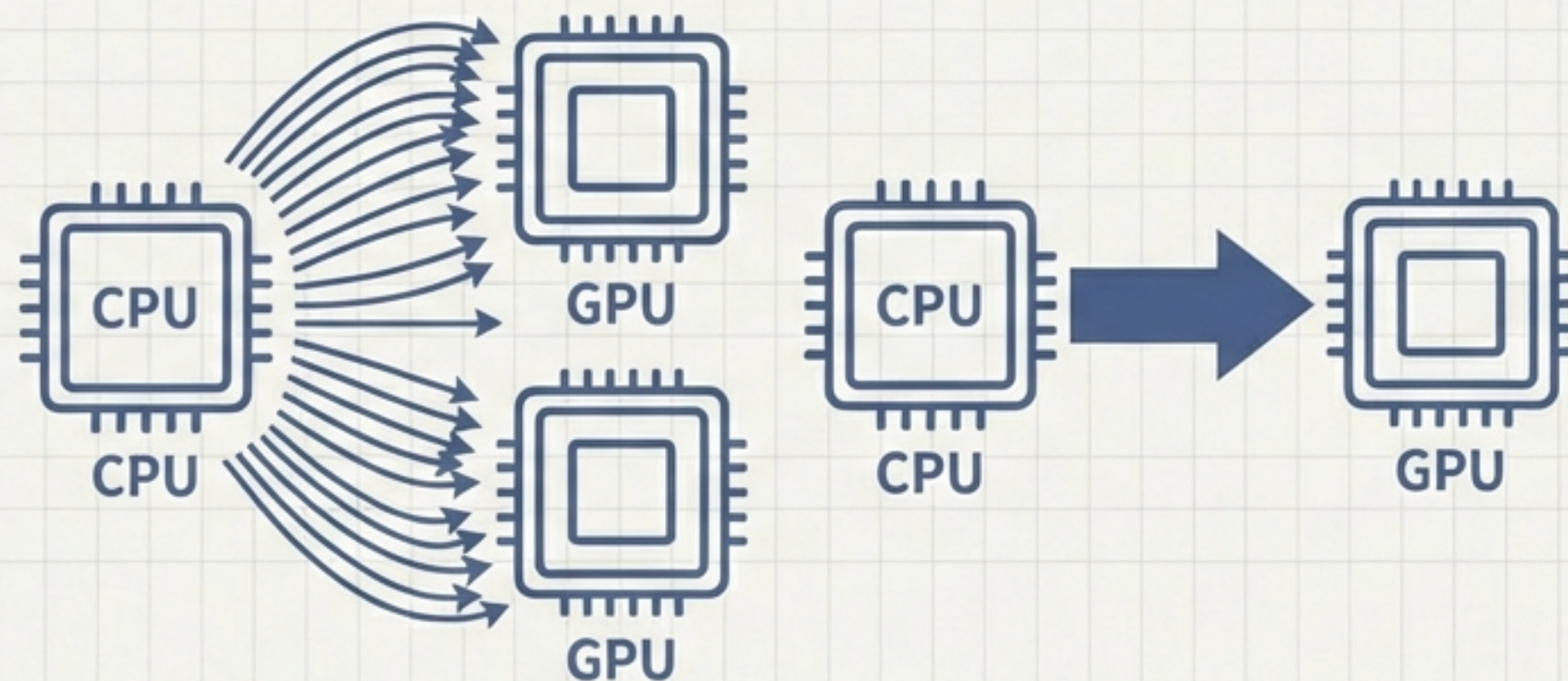
この判定があるからこそ、`activateTile()`を正しいタイルに対して呼び出すことができる。



## Annex B: 効率的な描画 — InstancedMeshでパフォーマンスを最大化する

数千ものタイルを個別の`Mesh`として描画すると、パフォーマンスが著しく低下します。

`InstancedMesh`は、同じジオメトリとマテリアルを持つ多数のオブジェクトを、単一の描画コールで処理する技術です。これにより、CPUの負荷が劇的に軽減されます。



非効率 (Inefficient)

効率的 (Efficient)

**Key Takeaway:** 大量のオブジェクトを扱う際は、`InstancedMesh`への切り替えを常に検討すべきです。



## Annex C: 設計のその先へ — あなた自身のゲームを創造する

技術的な基盤は整いました。ここから先は、あなたの創造性が試される領域です。どのようなルールや挑戦を加えることで、この世界はさらに面白くなるでしょうか？ 以下は、その出発点となるアイデアです。

- ☐ **ゴールを設定する (Set a Goal)** : 対岸の扉にたどり着く
- ☐ **時間制限を追加する (Add a Time Limit)** : 緊張感を高める
- ☐ **ランダム性を導入する (Introduce Randomness)** : 床の落下速度をタイルごとに変える
- ☐ **リスクとリターンを作る (Create Risk & Return)** : 走ると床が早く落ちる
- ☐ **プレイヤーを欺く (Deceive the Player)** : 一部の床を「落ちない」フェイクにする

この設計図は始まりに過ぎません。最高の建築家は、常に設計図を超えていくものです。